CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 13:
Protected Access/Friend Functions/Classes

Course taught by: Zubair Irshad

Courtesy of some slides: Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk
)

Unit Outline

1. Introduction
2. Protected Access
3. Friend Functions
4. Friend Classes
5. Summary

Form Motivation for Protected Access!

- Recall that in C++, in general there were access modifiers `public` and `private` that we have studied earlier.
- A `public` variable can be accessed and modified by any function within or outside of the class.
- For safety, in general we keep variables as '`private`' and provide access to them using accessor (**getter**) and mutator (**setter**) functions.
  - Recall the **Midterm coding component!**
- Likewise, functions, in general are declared as public, since these can be accessed by any function inside or outside of the class.
- However, private functions can also be declared.

Protected Access

- Another category of access level modifiers is **protected** access.
- Protected Access is in general, a feature of object oriented programming.
- Class members declared as **protected** can be accessed by any subclass (derived class) of that class as well.
- However, protected members cannot be accessed by classes outside of the inheritance hierarchy of the base class.

Protected Access – Example

```
#include <iostream>
using namespace std;
class Base {
protected:
    int protectedValue; // Protected member
public:
    Base() : protectedValue(0) {} // What is this called?
    void setValue(int val) {
        protectedValue = val;
    }
    void display() {
        cout << "Protected Value: " << protectedValue << endl;
}};
```

Protected Access – Example

```
class Derived : public Base {
public:
    void incrementValue() {
        protectedValue++; // Accessing protected member in subclass
}};
int main() {
    Derived obj;
    obj.setValue(10); // Accessible because it's public in Base
    obj.incrementValue(); // Indirectly modifies the protected member
    obj.display(); // Outputs: Protected Value: 11
    //obj.protectedValue = 20; //Cannot access protected member
directly, why?
    return 0;
}
```

Protected Access – Application/Uses

Applications/Uses

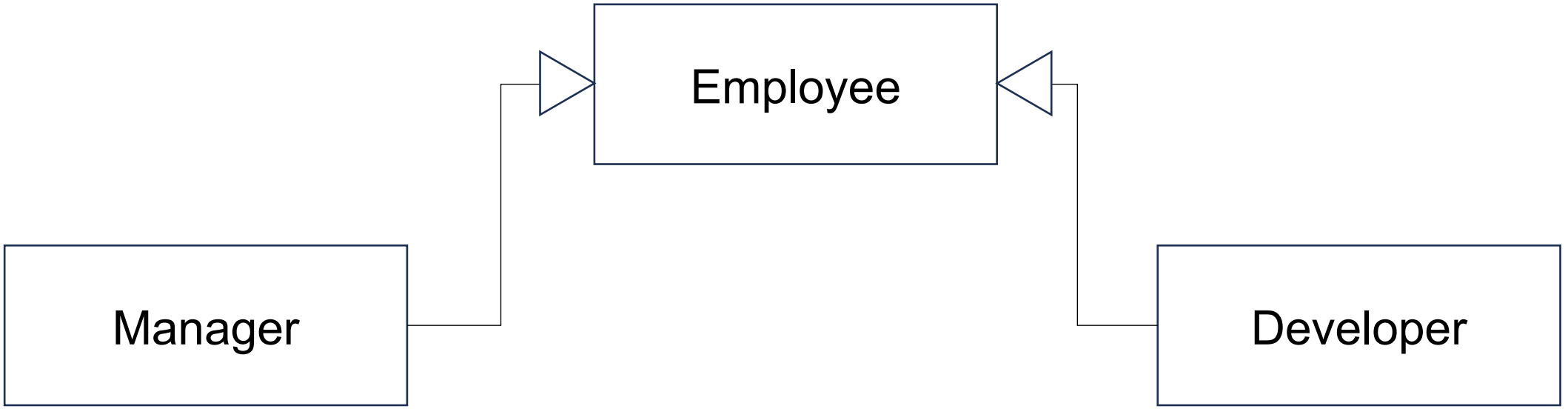
- **Protected Access** provides a middle ground between private (accessible only within the class) and public (accessible by everyone).
- It is often used to allow subclasses to use or modify inherited members without exposing those members to other parts of the program.

Benefits of Protected Access

- **Encapsulation with flexibility:** Members are hidden from the outside world but still available to subclasses.
- **Reusability:** Subclasses can reuse and extend the functionality of the base class while maintaining the integrity of its data.

Protected Access – Real World Example

- Class Demo
- UML Structure



Protected Access – Real World Example

```
// Base Class
class Employee {
protected:
    string name;
    double salary;
    int id;
public:
    // Constructor using initializer list
    Employee(string n, double s, int i)
        : name(n), salary(s), id(i) {}

    void displayInfo() {
        cout << "ID: " << id << " | Name: " << name
             << " | Salary: $" << salary << endl;
    }
};
```

Protected Access – Real World Example

```
// Derived Class 1
class Manager : public Employee {
private:
    int teamSize;
public:
    Manager(string n, double s, int i, int t)
        : Employee(n, s, i), teamSize(t) {} // Remember initializer list for
inheritance

    void giveRaise(double amount) {
        salary += amount;  // Allowed: salary is protected
    }

    void displayManager() {
        displayInfo();
        cout << "Team Size: " << teamSize << endl;
    }
};
```

Protected Access – Real World Example

```
// Derived Class 2
class Developer : public Employee {
private:
    string programmingLanguage;
public:
    Developer(string n, double s, int i, string lang)
        : Employee(n, s, i), programmingLanguage(lang) {}

    void promote() {
        salary += 2000;  // ✅ Accessing protected member
    }

    void displayDeveloper() {
        displayInfo();
        cout << "Primary Language: " << programmingLanguage << endl;
    }
};
```

Protected Access – Real World Example

```
// Main Function
int main() {
    Manager m("Alice", 80000, 101, 5);
    Developer d("Bob", 70000, 102, "C++");

    m.giveRaise(5000);
    d.promote();

    cout << "\n--- Employee Details ---\n";
    m.displayManager();
    d.displayDeveloper();

    // ❌ m.salary = 90000; // Not allowed: protected member
    return 0;
}
```

Friend Functions

- **Friend functions** are functions that are **not members of a class** but are **allowed access to the class's private and protected members**.
- They provide a way to extend the functionality of a class without making the function a member of the class itself.
- To declare a friend function, the keyword **`friend`** is used inside the class definition.

Friend Functions

- The concepts of encapsulation and data hiding dictate that nonmember functions **should not be able to** access an object's private or protected data.
- However, there are situations where such rigid discrimination leads to inefficiencies and awkward behavior.
- In this situation a **friend** function can act as a bridge between two classes.
- **Note/Hint for understanding:** Friend functions are **declared inside** but always **defined outside the class** i.e. they are **non-member functions!**
- Let's look at a few of these cases with examples:

Friend Functions and Static Functions – Recap!

- Note: Both mean very different things!
- **Static function** is a function that belongs to a **class** but **does not operate on a specific instance of that class.**
  - It is denoted with a **static** keyword!
  - Example of calling a static function:
    ```
    int result = MathUtils::add(5, 3);
    ```
- **Friend functions** are are **non-members functions of a class** but are **allowed access to the class's private and protected members**.
- They provide a way to extend the functionality of a class without making the function a member of the class itself.
- To declare a friend function, the keyword **`friend`** is used inside the class definition.

Friend Functions – Example – Same class

```
#include <iostream>
using namespace std;

class Box {
private:
    double length, width, height;

public:
    // Constructor
    Box(double l, double w, double h) :
    length(l), width(w), height(h) {}

    // Friend function declaration
    friend double calculateVolume(Box b);
};

// Friend function definition
double calculateVolume(Box b) {
    // Access private members directly
    return b.length * b.width *
b.height;
}

int main() {
    Box myBox(3.0, 4.0, 5.0);
    cout << "Volume of the box: " <<
calculateVolume(myBox) << endl;
    return 0;
}
```

Friend Functions – Example – Same class

- **Friend Function Access**
  - calculateVolume() is *not* a member of the Box class.
  - Yet, it can **access private members** (length, width, and height) directly because it was declared as a friend inside Box.
  - This *lifts* the restriction of private access **only for that function**.

- **Encapsulation Still Preserved**
  - Only calculateVolume() has this privilege — not *all* external functions.
  - You didn't have to make length, width, and height **public**, which keeps them protected from unintended changes.

Friend Functions – Example – Bridge 2 classes

```
#include <iostream>
using namespace std;

class beta;
//needed for frifunc declaration

class alpha
{
    private:
        int data;
    public:
        alpha() : data(3) { }
       //no-arg constructor
        friend int frifunc(alpha,
beta);
        //friend function
};
```

```
class beta
{
    private:
        int data;
    public:
        beta() : data(7) { }
        friend int frifunc(alpha,
beta);
};
int frifunc(alpha a, beta b)
//function definition
{
    return( a.data + b.data );
}
```

Friend Functions Example – Bridge 2 classes

```
//-----------------------------------------------------------
int main()
{
    alpha aa;
    beta bb;
    cout << frifunc(aa, bb) << endl;
    //call the function
    return 0;
}
```

OUTPUT:

10

Friend Functions – Bridge 2 classes Explanation!

- Normally, a function **outside** a class cannot access its private data.
  Here we saw **two separate classes** (alpha and beta), each with their own private data.
- If we wanted to write a single function that uses *both classes' private data*, it would normally be **impossible**, because neither class allows external access.
- An alternate, we can use **getter** and **setter** functions to access private data attributes – but seems to be a lot of lines of code!
- **The Solution – Friend Function:** Declare the **same function** as a friend in *both* classes.

Some Considerations

[Encapsulation]

- While friend functions can access private members, use them sparingly to maintain encapsulation principles.
- Overuse can lead to tightly coupled code.

[Efficiency]

- Friend functions can improve efficiency by eliminating the need for getters or setters for one-off operations.

[**Alternatives]**

- If a function is closely tied to a class, consider whether it should be a member instead of a friend.

Friend Classes

- In C++, **friend classes** are classes that are allowed to **access the private and protected members** of another class.
- **Declaring one class as a friend of another** enables tight coupling between the two classes.
- This allows the friend class to access and modify the internals of the other class directly.
- The friendship relationship is one-way, meaning if **ClassA** declares **ClassB** as a friend, **ClassB** can access **ClassA's private and protected members**, but not vice versa unless explicitly declared.

Friend Classes – Example

```
#include <iostream>
using namespace std;

class BoxInspector;

class Box {
private:
    double length, width, height; // Private members

public:
    // Constructor
    Box(double l, double w, double h) : length(l), width(w), height(h) {}

    // Declare BoxInspector as a friend class
    friend class BoxInspector;
};
```

Friend Classes – Example

```
// Friend class
class BoxInspector {
public:
    // Method to calculate the volume of the box
    double calculateVolume(const Box& box) {
        return box.length * box.width * box.height;
       // Accessing private members of Box
    }

    // Method to display the dimensions of the box
    void displayDimensions(const Box& box) {
        cout << "Box Dimensions: "
             << box.length << " x " << box.width << " x " << box.height <<
endl;
    }
};
```

Friend Classes – Example

```
int main() {
    Box myBox(3.0, 4.0, 5.0);
    BoxInspector inspector;

    // Use the friend class to inspect the Box
    inspector.displayDimensions(myBox);
    cout << "Volume: " << inspector.calculateVolume(myBox) << endl;

    return 0;
}
```

Why Use Friend Classes?

- **Encapsulation with Collaboration:**
  - The BoxInspector class operates on Box's private data without exposing that data directly to the outside world.
- **Cleaner Code:**
  - Eliminates the need for excessive getters and setters, which can clutter the Box class.
- **Real-World Use:**
  - Useful in scenarios where one class is a natural manager of another class's data but not really inheritance. If they are tightly coupled, think if you can use inheritance instead.
- This example shows how friend classes allow controlled interaction while keeping data encapsulated.

Summary

- Friend functions are non-member functions of a class but are allowed access to the class's private and protected members.
- Similarly friend classes are allowed access to private and protected data members.